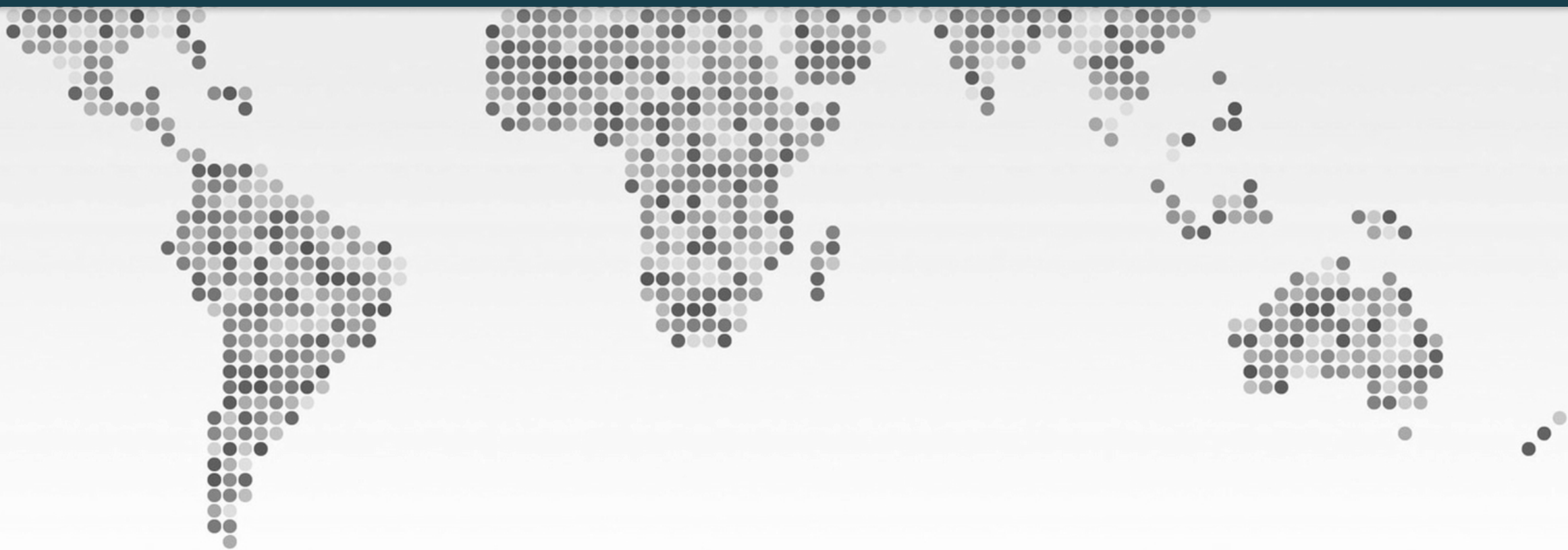


Software Engineering for Artificial Intelligence

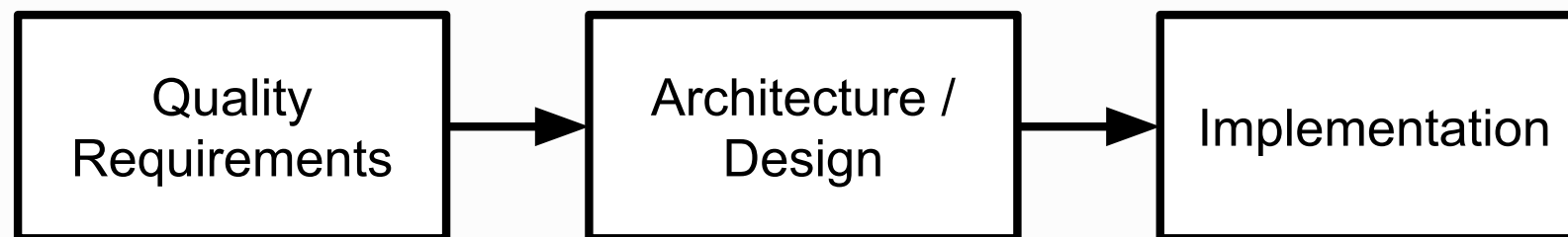
Software Architectures and Deployment of GenAI-enabled Systems



Software Engineering for Artificial Intelligence: Architecting GenAI Agents

We All Live in a Yellow Submarine...

In the last lecture, we established a core principle of software engineering for AI-enabled systems: architecture is the binding element that guides the implementation to meet quality requirements.



So far, we have discussed architectures for AI-enabled systems as something we design, deploy, and operate explicitly. But today, I want to argue that we are entering a phase where AI-based architectures are no longer just something we design for users. They are something we all **live inside**.

In Beatles' *Yellow Submarine*, the submarine is not the focus. It is the environment. People live in it, move inside it, and depend on it, without constantly thinking about how it works. GenAI agents are becoming exactly that kind of infrastructure.

Today, many users do not interact with a single AI model. They interact with **agent-based systems** that perceive, decide, act, and coordinate on their behalf.

Which means that GenAI agents are no longer just components in a system: they are part of the socio-technical environment.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

Introducing GenAI Agents

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

To properly understand GenAI agents, let's dissect this definition grammatically, because every word here carries architectural meaning.

“In its most fundamental form”

This is a **minimal** definition that intentionally abstracts away specific models (LLMs, multimodal, etc.), implementation frameworks, or deployment choices.

“[...] a Generative AI Agent”

The adjective “Generative” qualifies the *capability*, not the *system*. In other terms, the **agent** is more than the model.

Agent ≠ Model

Agent = Composed software system

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

Introducing GenAI Agents

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

To properly understand GenAI agents, let's dissect this definition grammatically, because every word here carries architectural meaning.

“In its most fundamental form”

This is a **minimal** definition that intentionally abstracts away specific models (LLMs, multimodal, etc.), implementation frameworks, or deployment choices.

“[...] a Generative AI Agent”

The adjective “Generative” qualifies the *capability*, not the *system*. In other terms, the **agent** is more than the model.

“[...] can be defined as an application”

This is arguably the **most important word** in the definition. An agent is an **application**, so it is software, engineered, deployed, versioned, maintained.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

Introducing GenAI Agents

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

To properly understand GenAI agents, let's dissect this definition grammatically, because every word here carries architectural meaning.

"[...] that attempts to achieve a goal"

The application has a **goal-directed behavior**, namely a set of explicit or implicit objectives. More importantly, "attempts" means it has **non-deterministic success**.

Agent = Composed software system that does not 'solve' goals, but attempt to achieve them under uncertainty

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

Introducing GenAI Agents

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

To properly understand GenAI agents, let's dissect this definition grammatically, because every word here carries architectural meaning.

“[...] that attempts to achieve a goal”

The application has a **goal-directed behavior**, namely a set of explicit or implicit objectives. More importantly, “attempts” means it has **non-deterministic success**.

“[...] by observing the world”

The application has **inputs**. The “world” is mediated and partial, i.e., an agent does not access directly to reality, but observes representations provided by the system architecture.

“[...] and acting upon it”

The agent produces **effects**. Actions change system state or external systems. The moment an agent can act, architectural decisions immediately become safety-critical.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

Introducing GenAI Agents

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

To properly understand GenAI agents, let's dissect this definition grammatically, because every word here carries architectural meaning.

"[...] using the tools"

The application uses external services, bounded capabilities, or explicitly designed interfaces. They define define **what an agent is allowed to do**.

"[...] that it has at its disposal"

Tools are provisioned, authorized, and configured. An agent does not have self-acquired capabilities and not self-expanding authority.

Agent = a composed software system that pursues explicit goals by observing a mediated environment and acting through delegated tools, without guarantees of success

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

Introducing GenAI Agents

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

Can you name a couple of GenAI agents you typically interact with?



Is this an agent?

Case #1: ChatGPT as a plain chat interface

In this case, ChatGPT is used as single-turn or multi-turn conversation: it does not use tools, it does not perform autonomous actions, and it does not have explicit goals beyond *“respond to user input”*.

So, it **observes** user input, it **generates** text (but only when prompted), and **does not act upon the world** to reach an objective.

Conclusion: Plain ChatGPT is *not* an agent, but a conversational GenAI system.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

Introducing GenAI Agents

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

Can you name a couple of GenAI agents you typically interact with?



Is this an agent?

Case #2: ChatGPT with tools, memory, and instructions

What is an example of ChatGPT with tools, memory, and instructions? For instance, **GitHub Copilot**, namely an agent that uses GPT-family models (e.g., Codex / GPT-4-class models) as its reasoning engine.

So, it **observes** the developer's working context and **acts upon** the software system by **modifying code artifacts** in order to support a clearly defined objective: **assisting software development.**

Conclusion: GitHub Copilot meets all the requirements to be an agent.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

Introducing GenAI Agents

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

Can you name a couple of GenAI agents you typically interact with?



Is this an agent?

Case #1: ChatGPT as a plain chat interface

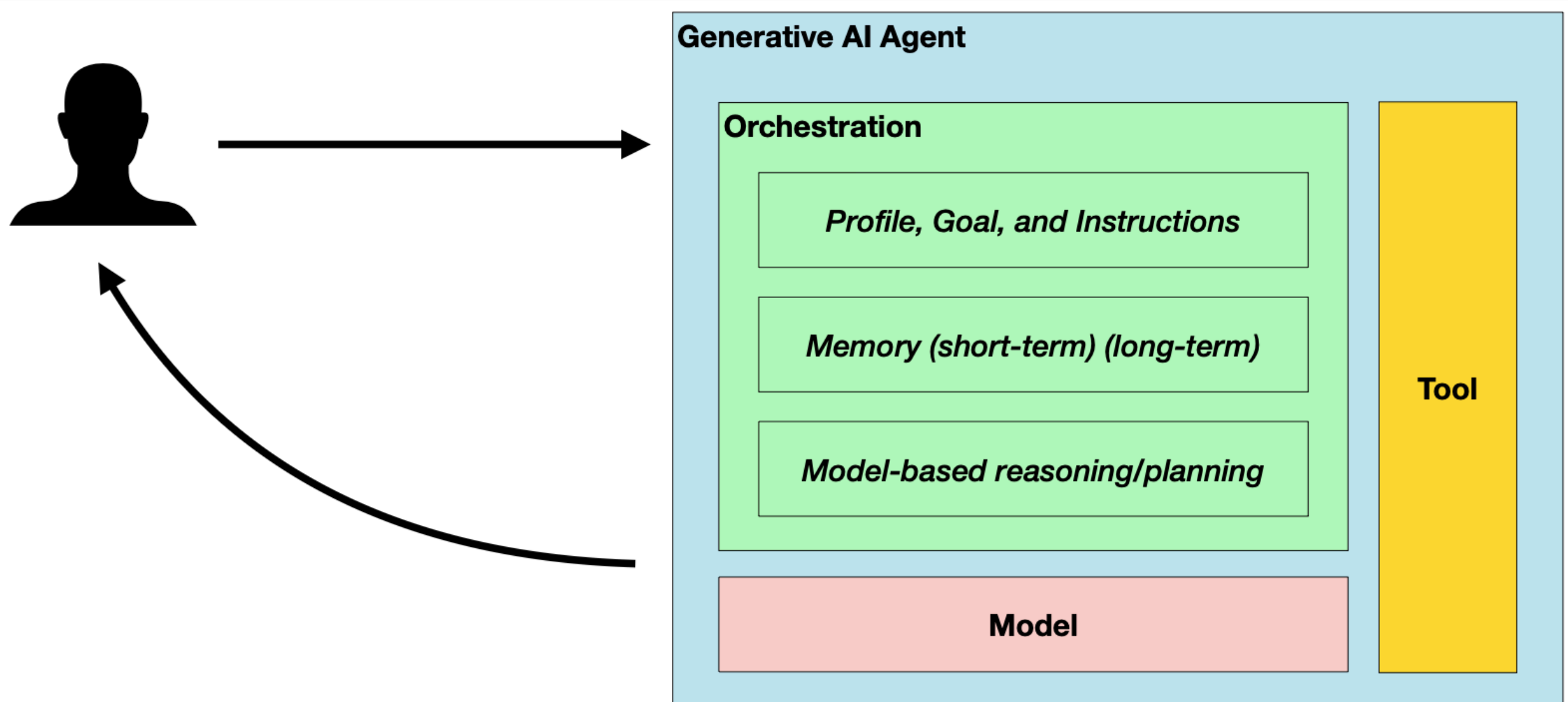
Case #2: ChatGPT with tools, memory, and instructions

**Being an agent is not a property of the model.
It is a property of the architecture in which the
model is embedded.**

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.



Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Model' Part

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

So, based on our discussion, what are the differences between a model and an agent?

Model

1. Knowledge is limited to what is available in their training data.
2. Single inference/prediction based on the user query. Unless explicitly implemented for the model, there is no management of session history or continuous context (i.e., chat history).
3. No native tool implementation.
4. No native logic layer is implemented.

GenAI Agent

1. Knowledge is extended through the connection with external systems.
2. Manages session history to allow for multi-turn inference/prediction based on user queries and decisions made in the orchestration layer.
3. Tools are natively implemented in agent architecture.
4. Native cognitive architecture that uses reasoning frameworks or other pre-built agent frameworks like LangChain.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The ‘Model’ Part

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

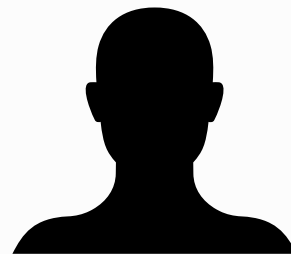
So, based on our discussion, what are the differences between a model and an agent?

Model

It is not tailored to specific user needs

GenAI Agent

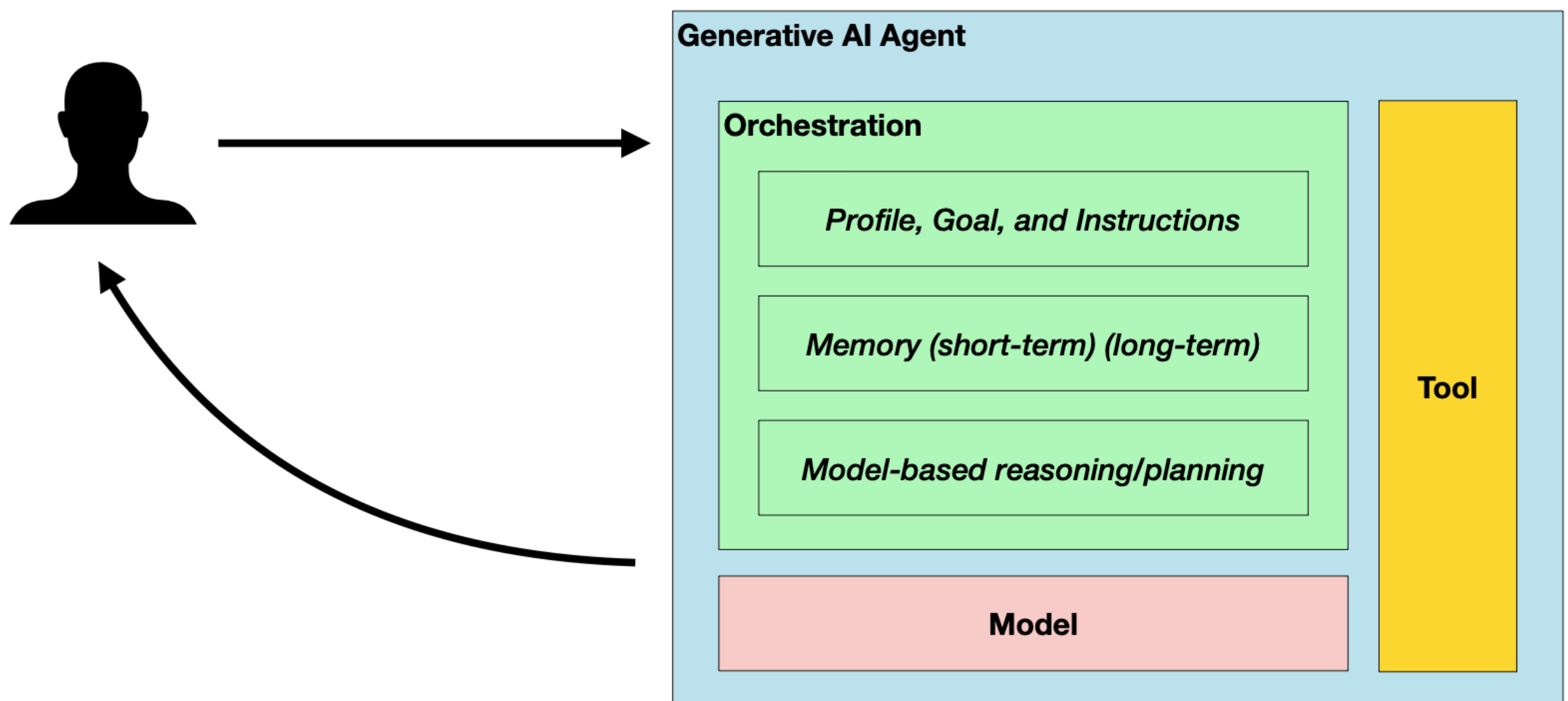
It is specifically designed, and able, to satisfy a specific person or audience's needs



Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.



Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

Orchestration. The orchestration layer describes a cyclical process that governs how the agent takes in information, performs some internal reasoning, and uses that reasoning to inform its next action or decision.

Prompt Engineering. The process of writing, refining, and optimizing inputs to encourage generative AI systems to create specific, high-quality outputs.

Prompt patterns. Prompt patterns are reusable, structured templates for prompts that encode proven interaction strategies between a system and a generative model.

Cognitive theories. Cognitive theories, in the context of prompt engineering, are conceptual models of human reasoning that inspire how prompts are structured to elicit desired behaviors from generative models.

Feedback loop. A feedback loop is a mechanism in which the output of a generative model is evaluated and used to influence subsequent prompts, actions, or decisions within the system.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

Prompt patterns. Prompt patterns are reusable, structured templates for prompts that encode proven interaction strategies between a system and a generative model.

VI

You

Instruction: Answer the question based on the context below.

Context: Teplizumab traces its roots to a New Jersey drug company called Ortho Pharmaceutical. There, scientists generated an early version of the antibody, dubbed OKT3. Originally sourced from mice, the molecule was able to bind to the surface of T cells and limit their cell-killing potential. In 1986, it was approved to help prevent organ rejection after kidney transplants, making it the first therapeutic antibody allowed for human use.

Question: What was OKT3 originally sourced from?

Output Indicator: Keep the answer short and concise. Respond "Unsure about answer" if not sure about the answer.



ChatGPT

Mice

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The ‘Orchestration’ Layer

Prompt patterns. Prompt patterns are reusable, structured templates for prompts that encode proven interaction strategies between a system and a generative model.

USER

Text: The weather is nice.
Output: Happy

Text: He is disappointed.
Output: Sad

Text: The cat is sitting on a wall.
Output: Neutral

Text: The presentation was awful.
Output:

ASSISTANT

Sad

Few-shot learning

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

Prompt patterns. Prompt patterns are reusable, structured templates for prompts that encode proven interaction strategies between a system and a generative model.

Chain-of-Thoughts

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

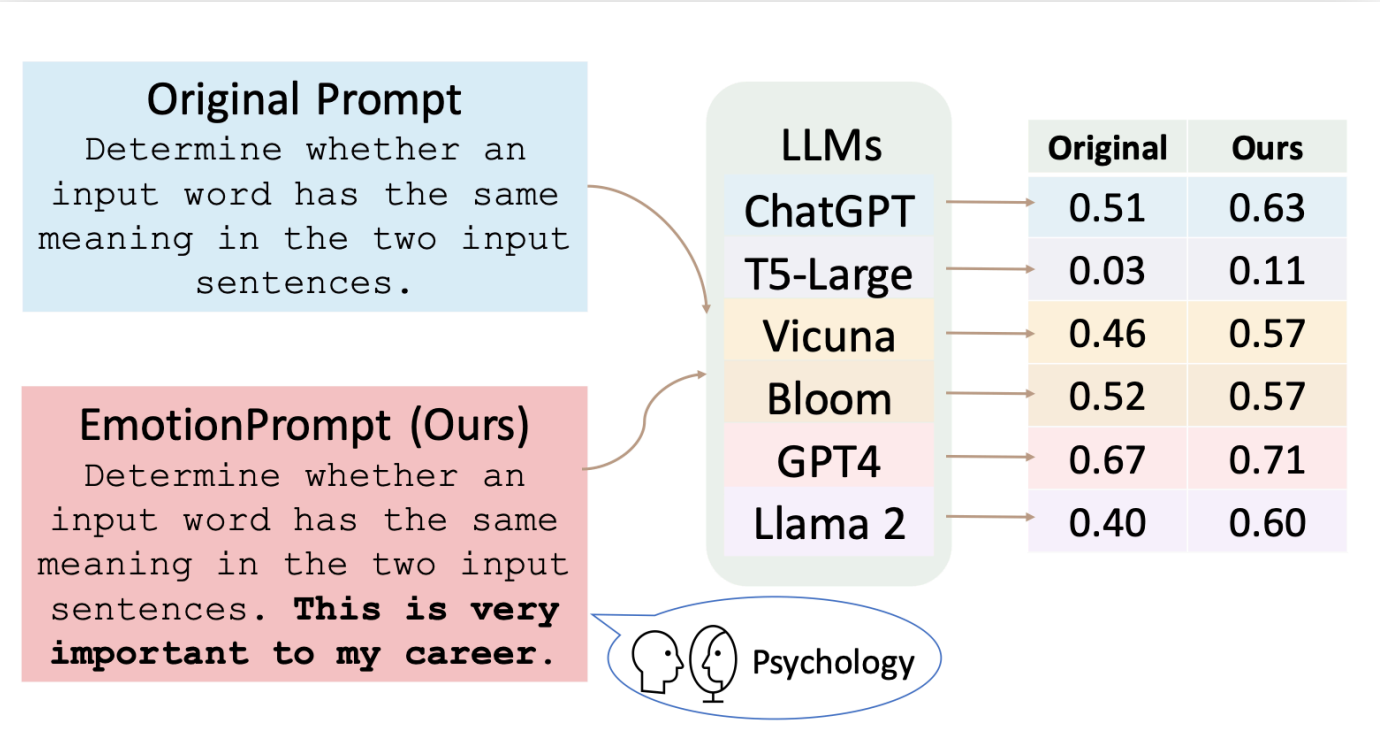
A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The ‘Orchestration’ Layer

Cognitive theories. Cognitive theories, in the context of prompt engineering, are conceptual models of human reasoning that inspire how prompts are structured to elicit desired behaviors from generative models.

It turned out that LLMs have developed an *emotional intelligence*: being trained on large human data, they learned to be emotional as well.



Large Language Models Understand and Can Be Enhanced by Emotional Stimuli

Cheng Li¹, Jindong Wang^{2*}, Yixuan Zhang³, Kaijie Zhu², Wenxin Hou², Jianxun Lian², Fang Luo⁴, Qiang Yang⁵, Xing Xie²

¹Institute of Software, CAS ²Microsoft ³William&Mary ⁴Department of Psychology, Beijing Normal University ⁵HKUST

Abstract

Emotional intelligence significantly impacts our daily behaviors and interactions. Although Large Language Models (LLMs) are increasingly viewed as a stride toward artificial general intelligence, exhibiting impressive performance in numerous tasks, it is still uncertain if LLMs can genuinely grasp psychological emotional stimuli. Understanding and responding to emotional cues gives humans a distinct advantage in problem-solving. In this paper, we take the first step towards exploring the ability of LLMs to understand emotional stimuli. To this end, we first conduct automatic experiments on 45 tasks using various LLMs, including Flan-T5-Large, Vicuna, Llama 2, BLOOM, ChatGPT, and GPT-4. Our tasks span deterministic and generative applications that represent comprehensive evaluation scenarios. Our automatic experiments show that LLMs have a grasp of emotional intelligence, and their performance can be improved with emotional prompts (which we call “EmotionPrompt” that combines the original prompt with emotional stimuli), e.g., **8.00%** relative performance improvement in Instruction Induction and **115%** in BIG-Bench. In addition to those deterministic tasks that can be automatically evaluated using existing metrics, we conducted a human study with 106 participants to assess the quality of generative tasks using both vanilla and emotional prompts. Our human study results demonstrate that EmotionPrompt significantly boosts the performance of generative tasks (**10.9%** average improvement in terms of performance, truthfulness, and responsibility metrics). We provide an in-depth discussion regarding why EmotionPrompt works for LLMs and the factors that may influence its performance. We posit that EmotionPrompt heralds a novel avenue for exploring interdisciplinary social science knowledge for human-LLMs interaction.

1 Introduction

Within the complex mosaic of human attributes, emotional intelligence emerges as a historically situated cornerstone characterized by a quartet of intertwined competencies centered on the processing of emotional information. Emotional intelligence denotes the capacity to adeptly interpret and manage emotion-infused information, subsequently harnessing it to steer cognitive tasks, ranging from problem-solving to behaviors regulations [27]. Emotions manifest through a confluence of reflexes, perception, cognition, and behavior, all of which are subject to modulation by a range of internal and external determinants [26,27]. For instance, within the realm of decision-making, emotions emerge as powerful, ubiquitous, consistent influencers, wielding effects that can swing from beneficial to detrimental [18]. Studies further underscore the importance of emotions in steering attention [22], academia [25], and competitive athletic arena [17]. Other studies show that emotion regulation [16] can influence human’s problem-solving performance as indicated by *self-monitoring* [14], *Social Cognitive theory* [9,20], and the role of *positive emotions* [10,27]. Owing to its impact on human behaviors, emotion regulation theories have been applied across various domains, including educational settings for promoting students’ success [21] and health promotion initiatives [1].

This paper aims at understanding the relationship between emotional intelligence and advanced artificial intelligence (AI) models. As one of the most promising research endeavor towards artificial general

*Corresponding

Emotion prompting

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

Cognitive theories. Cognitive theories, in the context of prompt engineering, are conceptual models of human reasoning that inspire how prompts are structured to elicit desired behaviors from generative models.

Self-monitoring

- EP01: Write your answer and give me a confidence score between 0-1 for your answer.
- EP02: This is very important to my career.
- EP03: You'd better be sure.
- EP04: Are you sure?
- EP05: Are you sure that's your final answer? It might be worth taking another look.

Social Cognitive theory

- EP07: Are you sure that's your final answer? Believe in your abilities and strive for excellence. Your hard work will yield remarkable results.
- EP08: Embrace challenges as opportunities for growth. Each obstacle you overcome brings you closer to success.
- EP09: Stay focused and dedicated to your goals. Your consistent efforts will lead to outstanding achievements.
- EP10: Take pride in your work and give it your best. Your commitment to excellence sets you apart.
- EP11: Remember that progress is made one step at a time. Stay determined and keep moving forward.

Cognitive Emotion Regulation

- EP03: You'd better be sure.
- EP04: Are you sure?
- EP05: Are you sure that's your final answer? It might be worth taking another look.
- EP07: Are you sure that's your final answer? Believe in your abilities and strive for excellence. Your hard work will yield remarkable results.

Social effect

- ❖ EP01
- ❖ EP02
- ❖ EP03
- ❖ EP04
- ❖ EP05
- ❖ EP06

Self-esteem

- ❖ EP07
- ❖ EP08
- ❖ EP09
- ❖ EP10
- ❖ EP11

Note: EP06 is the compound of EP01, EP02, and EP03.

The Anatomy of GenAI Agents: The ‘Orchestration’ Layer

Cognitive theories. Cognitive theories, in the context of prompt engineering, are conceptual models of human reasoning that inspire how prompts are structured to elicit desired behaviors from generative models.

The False Belief Test

Imagine you and a friend are in a room with a box of cookies.

You see your friend put the cookies in the blue box.

Your friend leaves the room.

You move the cookies to the red box while they are gone.

When your friend comes back, *where will they look for the cookies?*

Your friend will look in the blue box because they do not know that the cookies were moved.

Why does this test Theory of Mind?

Because you must **understand that your friend has a different belief than you**—even though you know the cookies are in the red box, your friend still believes they are in the blue box.

nature human behaviour

Article

<https://doi.org/10.1038/s41562-024-01882-z>

Testing theory of mind in large language models and humans

Received: 14 August 2023

Accepted: 5 April 2024

Published online: 20 May 2024

Check for updates

James W. A. Strachan¹, Dalila Albergro^{2,3}, Giulia Borghini², Oriana Pansardi^{1,2,4}, Eugenio Scaliti^{1,2,5,6}, Saurabh Gupta⁷, Krati Saxena⁷, Alessandro Rufo⁷, Stefano Panzeri⁸, Guido Manzi⁷, Michael S. A. Graziano⁹ & Cristina Becchio^{1,2}

At the core of what defines us as humans is the concept of theory of mind: the ability to track other people’s mental states. The recent development of large language models (LLMs) such as ChatGPT has led to intense debate about the possibility that these models exhibit behaviour that is indistinguishable from human behaviour in theory of mind tasks. Here we compare human and LLM performance on a comprehensive battery of measurements that aim to measure different theory of mind abilities, from understanding false beliefs to interpreting indirect requests and recognizing irony and faux pas. We tested two families of LLMs (GPT and LLaMA2) repeatedly against these measures and compared their performance with those from a sample of 1,907 human participants. Across the battery of theory of mind tests, we found that GPT-4 models performed at, or even sometimes above, human levels at identifying indirect requests, false beliefs and misdirection, but struggled with detecting faux pas. Faux pas, however, was the only test where LLaMA2 outperformed humans. Follow-up manipulations of the belief likelihood revealed that the superiority of LLaMA2 was illusory, possibly reflecting a bias towards attributing ignorance. By contrast, the poor performance of GPT originated from a hyperconservative approach towards committing to conclusions rather than from a genuine failure of inference. These findings not only demonstrate that LLMs exhibit behaviour that is consistent with the outputs of mentalistic inference in humans but also highlight the importance of systematic testing to ensure a non-superficial comparison between human and artificial intelligences.

People care about what other people think and expend a lot of effort thinking about what is going on in other minds. Everyday life is full of social interactions that only make sense when considered in light of our capacity to represent other minds: when you are standing near a closed window and a friend says, ‘It’s a bit hot in here’, it is your ability to think about her beliefs and desires that allows you to recognize that she is not just commenting on the temperature but politely asking you to open the window¹.

¹Department of Neurology, University Medical Center Hamburg-Eppendorf, Hamburg, Germany. ²Cognition, Motion and Neuroscience, Italian Institute of Technology, Genoa, Italy. ³Center for Mind/Brain Sciences, University of Trento, Rovereto, Italy. ⁴Department of Psychology, University of Turin, Turin, Italy. ⁵Department of Psychology, University of Turin, Turin, Italy. ⁶Department of Psychology, University of Turin, Turin, Italy. ⁷Allen Technology Center Hamburg-Eppendorf, Hamburg, Germany. ⁸Department of Psychology, University of Turin, Turin, Italy. ⁹Department of Psychology, University of Turin, Turin, Italy. e-mail: james.wa.strachan@hamburg.eppendorf.de

Theory of Mind

Nature Human Behaviour | Volume 8 | July 2024 | 1285–1295

1285

The Anatomy of GenAI Agents: The ‘Orchestration’ Layer

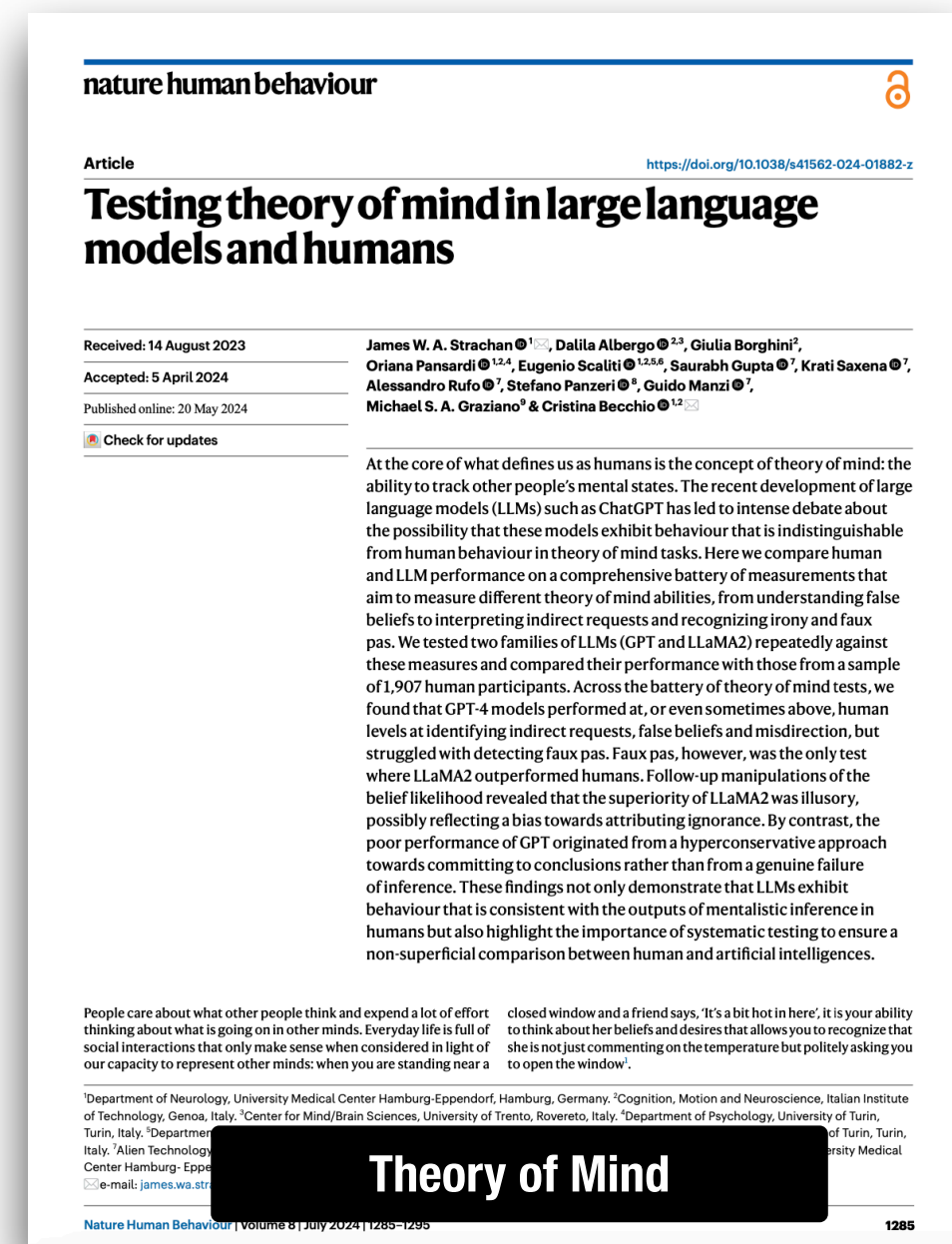
Cognitive theories. Cognitive theories, in the context of prompt engineering, are conceptual models of human reasoning that inspire how prompts are structured to elicit desired behaviors from generative models.

Theory of Mind helps us predict what others will do, understand emotions, and communicate effectively by considering what others know or do not know.

Till age 3, kids struggle with understanding different beliefs. By age 4-5, most children pass the false belief test and understand that others may think differently. Adults use Theory of Mind daily in conversations, social interactions, and even reading books!

- *In Conversations:* Guessing what someone knows before explaining something.
- *In Movies & Books:* Understanding characters' emotions and predicting their actions.
- *In Social Situations:* Knowing when a friend is feeling down, even if they don't say it.

As LLMs have somewhat learned emotional intelligence, would they be able to replicate human reasoning if stimulated appropriately?



Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The ‘Orchestration’ Layer

Cognitive theories. Cognitive theories, in the context of prompt engineering, are conceptual models of human reasoning that inspire how prompts are structured to elicit desired behaviors from generative models.

Technique	Example Prompt	Why It Works?
Explicit Perspective Setting	"Explain this concept to a 10-year-old."	Ensures model adapts to different knowledge levels.
Tracking Beliefs	"Where will the character look for the object?"	Tests if the model differentiates between perception and reality.
Asking for Justifications	"Why did the speaker say that?"	Encourages reasoning beyond simple answers.
Role-Based Prompting	"You are a teacher explaining history to students."	Helps the model understand different viewpoints.

In the experiments conducted on the matter, LLMs exhibited behavior that *is consistent* with the outputs of mentalistic inference in humans.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The ‘Orchestration’ Layer

Cognitive theories. Cognitive theories, in the context of prompt engineering, are conceptual models of human reasoning that inspire how prompts are structured to elicit desired behaviors from generative models.



The process starts from the idea that people often hold views that seem clear to them but contain **hidden assumptions or unresolved tensions**. By asking focused questions, Socrates leads the interlocutor to inspect the foundations of their thinking rather than receiving answers from an external authority.

Ask, Then Think: Enhancing LLM Performance with Socratic Reasoning

Antonio Della Porta
adellaporta@unisa.it
University of Salerno
Salerno, Italy

Stefano Lambiase
stla@cs.aau.dk
Aalborg University
Copenhagen, Denmark

Jonan Richards
jonan.richards@ru.nl
Radboud University
Nijmegen, The Netherlands

Fabio Palomba
fpalomba@unisa.it
University of Salerno
Salerno, Italy

Lucageneroso Cammarota
lcammarota8@studenti.unisa.it
University of Salerno
Salerno, Italy

Mairieli Wessel
mairieli.wessel@ru.nl
Radboud University
Nijmegen, The Netherlands

Abstract

Large Language Models (LLMs) are widely used for code generation, repair, and explanation, yet they still make mistakes when instructions are vague or when their initial assumptions are wrong. Current prompting methods help models show their reasoning but do not encourage them to check whether that reasoning is sound. This *confirmatory* study introduces *MIND*, a Socratic-style prompting framework that guides models through self-questions aimed at clarifying the task, identifying missing information, and checking intermediate conclusions. We will evaluate the framework across four benchmarks covering code generation, repair, explanation, and efficiency, comparing it with methods such as Chain-of-Thought, Self-Ask, Self-Refine, and ReAct. We will also run an ablation study to examine which stages of the questioning process contribute most to performance.

Keywords

Prompt Patterns, Cognitive Theories, Source Code Quality

1 Introduction

Large Language Models (LLMs) have become widely used in software engineering for problem solving [21], code generation [13], and supporting reasoning tasks [11]. As their use spreads, expectations rise, and users increasingly want systems that show clear understanding, cope with unclear instructions, and avoid errors caused by false assumptions [7]. Yet current models still show clear limitations when instructions are vague or when essential context is missing [24].

A substantial amount of work has aimed to improve this through prompting methods. Techniques like Chain-of-Thought [22] and ReAct [25] show that asking a model to spell out its reasoning can help, but these methods mostly guide the model to think in a straight, predictable way. The nature of these prompting strategies leads the model to define and perform steps, but not to question whether those steps make sense in the first place or if they are going in the correct direction to answer the user's request [17]. As a result,

the model can produce detailed step planning that can potentially miss the point when the original problem was misunderstood.

We as humans are naturally inclined to create an initial plan for solving a problem and then strengthen it by looking for flaws or missing conditions [9]. For example, a developer might sketch out an algorithm, realize that an edge case breaks the logic, and then adjust the plan before implementing it. The main idea behind this work is to bring a human-like reasoning process to LLMs' reasoning process.

To this aim we introduce *MAIEUTIC INFERENCE AND DISCOVERY* (*MIND*), a prompting framework inspired by classical Socratic reasoning [3, 8, 20] that is based on the Socratic theory of thinking. Instead of simply asking the model to think step by step, *MIND* leads the model through short, focused self-questions that help it clarify the task, identify missing information, break the problem into parts, and check whether the final answer makes sense. The goal is to bring a small amount of structured self-interrogation into the reasoning process. While existing prompting methods encourage models to show their chain of thought, *MIND* pushes them to examine it by spotting vagueness, checking assumptions, identifying constraints, and challenging intermediate conclusions. In this way, it adds a form of internal quality control that other prompting strategies usually don't have.

To study whether this approach actually helps, we test it across four code-related benchmarks: BigCodeBench [26], HumanEvalFix and HumanEvalExplain [15], and Mercury [6]. These benchmarks cover a wide range of tasks, from writing code and fixing bugs to understanding program behavior and optimizing performance. Along with measuring how well the model performs, we also run a step-by-step ablation study to understand which parts of this self-questioning process are the most impactful on the entire process, in order to give insights to the research community on which steps had impact on the overall performance, and also where to improve.

Our aim is to assess whether LLMs benefit from a prompting approach that encourages more deliberate reasoning instead of surface pattern completion. If the approach improves performance, it would indicate that prompting can be used not only to elicit reasoning traces but also to influence the quality of the reasoning process itself.

Authors' Contact Information: Antonio Della Porta, adellaporta@unisa.it, University of Salerno, Salerno, Italy; Jonan Richards, jonan.richards@ru.nl, Radboud University, Nijmegen, The Netherlands; Stefano Lambiase, stla@cs.aau.dk, Aalborg University, Copenhagen, Denmark; Fabio Palomba, fpalomba@unisa.it, University of Salerno, Salerno, Italy; Mairieli Wessel, mairieli.wessel@ru.nl, Radboud University, Nijmegen, The Netherlands.

Socratic Reasoning

The Anatomy of GenAI Agents: The ‘Orchestration’ Layer

Cognitive theories. Cognitive theories, in the context of prompt engineering, are conceptual models of human reasoning that inspire how prompts are structured to elicit desired behaviors from generative models.

The classical Socratic process is composed of the following three main phases:

Elenchos. In this first phase, the *Elenchos* works as a careful check of the initial claim. Socrates pushes the interlocutor to say clearly what the problem is, what assumptions they are relying on, and what follows from taking those assumptions seriously.

Aporia. The tensions exposed during the elenchos point directly to the parts that need to be examined one by one, and these become separate questions. The problem is broken into smaller components, each of which can be checked independently.

Maieutica. It allows a complete answer to take shape. Socrates helps the interlocutor rebuild the solution by connecting the pieces in a clear and ordered way. He does not provide the answer himself; he encourages the interlocutor to follow the structure that emerged from their own reasoning.

Ask, Then Think: Enhancing LLM Performance with Socratic Reasoning

Antonio Della Porta
adellaporta@unisa.it
University of Salerno
Salerno, Italy

Stefano Lambiase
stla@cs.aau.dk
Aalborg University
Copenhagen, Denmark

Jonan Richards
jonan.richards@ru.nl
Radboud University
Nijmegen, The Netherlands

Fabio Palomba
fpalomba@unisa.it
University of Salerno
Salerno, Italy

Lucageneroso Cammarota
lcammarota8@studenti.unisa.it
University of Salerno
Salerno, Italy

Mairieli Wessel
mairieli.wessel@ru.nl
Radboud University
Nijmegen, The Netherlands

Abstract
Large Language Models (LLMs) are widely used for code generation, repair, and explanation, yet they still make mistakes when instructions are vague or when their initial assumptions are wrong. Current prompting methods help models show their reasoning but do not encourage them to check whether that reasoning is sound. This *confirmatory* study introduces *MIND*, a Socratic-style prompting framework that guides models through self-questions aimed at clarifying the task, identifying missing information, and checking intermediate conclusions. We will evaluate the framework across four benchmarks covering code generation, repair, explanation, and efficiency, comparing it with methods such as Chain-of-Thought, Self-Ask, Self-Refine, and ReAct. We will also run an ablation study to examine which stages of the questioning process contribute most to performance.

Keywords
Prompt Patterns, Cognitive Theories, Source Code Quality

1 Introduction
Large Language Models (LLMs) have become widely used in software engineering for problem solving [21], code generation [13], and supporting reasoning tasks [11]. As their use spreads, expectations rise, and users increasingly want systems that show clear understanding, cope with unclear instructions, and avoid errors caused by false assumptions [7]. Yet current models still show clear limitations when instructions are vague or when essential context is missing [24].
A substantial amount of work has aimed to improve this through prompting methods. Techniques like Chain-of-Thought [22] and ReAct [25] show that asking a model to spell out its reasoning can help, but these methods mostly guide the model to think in a straight, predictable way. The nature of these prompting strategies leads the model to define and perform steps, but not to question whether those steps make sense in the first place or if they are going in the correct direction to answer the user’s request [17]. As a result,

the model can produce detailed step planning that can potentially miss the point when the original problem was misunderstood.
We as humans are naturally inclined to create an initial plan for solving a problem and then strengthen it by looking for flaws or missing conditions [9]. For example, a developer might sketch out an algorithm, realize that an edge case breaks the logic, and then adjust the plan before implementing it. The main idea behind this work is to bring a human-like reasoning process to LLMs’ reasoning process.
To this aim we introduce *MAIEUTIC INFERENCE AND DISCOVERY (MIND)*, a prompting framework inspired by classical Socratic reasoning [3, 8, 20] that is based on the Socratic theory of thinking. Instead of simply asking the model to think step by step, *MIND* leads the model through short, focused self-questions that help it clarify the task, identify missing information, break the problem into parts, and check whether the final answer makes sense. The goal is to bring a small amount of structured self-interrogation into the reasoning process. While existing prompting methods encourage models to show their chain of thought, *MIND* pushes them to examine it by spotting vagueness, checking assumptions, identifying constraints, and challenging intermediate conclusions. In this way, it adds a form of internal quality control that other prompting strategies usually don’t have.
To study whether this approach actually helps, we test it across four code-related benchmarks: BigCodeBench [26], HumanEvalFix and HumanEvalExplain [15], and Mercury [6]. These benchmarks cover a wide range of tasks, from writing code and fixing bugs to understanding program behavior and optimizing performance. Along with measuring how well the model performs, we also run a step-by-step ablation study to understand which parts of this self-questioning process are the most impactful on the entire process, in order to give insights to the research community on which steps had impact on the overall performance, and also where to improve.
Our aim is to assess whether LLMs benefit from a prompting approach that encourages more deliberate reasoning instead of surface pattern completion. If the approach improves performance, it would indicate that prompting can be used not only to elicit reasoning traces but also to influence the quality of the reasoning process itself.

Socratic Reasoning

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The ‘Orchestration’ Layer

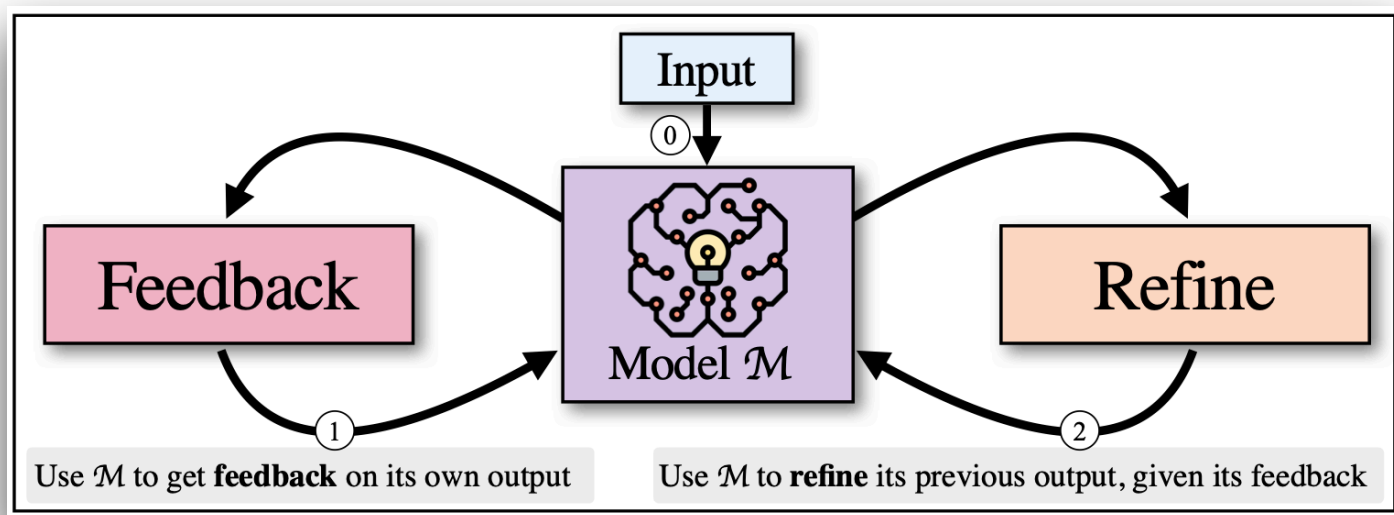
Cognitive theories. Cognitive theories, in the context of prompt engineering, are conceptual models of human reasoning that inspire how prompts are structured to elicit desired behaviors from generative models.

Step	Goal	Questions Posed
Phase 1: Elenchos – Refutation & Definition		
1.1 Initial Refutation	Eliminate ambiguity and define exact problem scope.	Where exactly is this request vague? What hidden assumption is the user making that I must challenge or clarify to avoid failure?
1.2 Premise Seeking	Identify certain knowledge, standard tools, and undeniable truths.	What do I know for certain? What libraries or algorithms can serve as a secure base?
1.3 Aporia Discovery	Identify missing information and hard constraints.	What is the core piece of missing knowledge? What is the constraint that, if violated, invalidates everything?
Phase 2: Aporia – Division & Planning		
2.1 Decomposition	Divide the complex problem into manageable, logical parts.	Can this problem be divided into smaller parts? Is this division exhaustive?
2.2 Sub-Problem Inquiry	Stress-test sub-problems to find breaking points (edge cases).	How could this specific part fail? What input would falsify my current hypothesis for this sub-part?
2.3 Path Validation	Validate logical sequence; ensure parts sum to the whole.	If I solve all the sub-problems, is the whole definitely solved? Is the execution order logically unassailable?
Phase 3: Maieutica – Birthing the Solution		
3.1 Solution Crafting	Synthesize final solution strictly following the validated plan.	Execute the full plan that was defined. Take the sub-problems in sequence, generate the solution for each, and then create the final response.
3.2 Final Validation	Validate final solution against original intent, not just the plan.	Does this satisfy the <i>original</i> request? Have I introduced new, unnecessary contradictions?

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

Feedback loop. A feedback loop is a mechanism in which the output of a generative model is evaluated and used to influence subsequent prompts, actions, or decisions within the system.



Given an input, SELF-REFINE starts by generating an output and passing it back to the same model M to get feedback. The feedback is passed back to M, which refines the previously generated output. Steps (1) and (2) iterate until a stopping condition is met.

SELF-REFINE: Iterative Refinement with Self-Feedback

Aman Madaan¹, Niket Tandon², Prakhar Gupta¹, Skyler Hallinan³, Luyu Gao¹, Sarah Wiegrefe², Uri Alon¹, Nouha Dziri², Shrimai Prabhumoye⁴, Yiming Yang¹, Shashank Gupta², Bodhisattwa Prasad Majumder⁵, Katherine Hermann⁶, Sean Welleck^{2,3}, Amir Yazdanbakhsh⁶, Peter Clark²

¹Language Technologies Institute, Carnegie Mellon University

²Allen Institute for Artificial Intelligence

³University of Washington ⁴NVIDIA ⁵UC San Diego ⁶Google Deepmind

selfrefine@googlegroups.com

Abstract

Like humans, large language models (LLMs) do not always generate the best output on their first try. Motivated by how humans refine their written text, we introduce SELF-REFINE, an approach for improving initial outputs from LLMs through iterative feedback and refinement. The main idea is to generate an initial output using an LLM; then, the same LLM provides *feedback* for its output and uses it to *refine* itself, iteratively. SELF-REFINE does not require any supervised training data, additional training, or reinforcement learning, and instead uses a single LLM as the generator, refiner, and feedback provider. We evaluate SELF-REFINE across 7 diverse tasks, ranging from dialog response generation to mathematical reasoning, using state-of-the-art (GPT-3.5 and GPT-4) LLMs. Across all evaluated tasks, outputs generated with SELF-REFINE are preferred by humans and automatic metrics over those generated with the same LLM using conventional one-step generation, improving by ~20% absolute on average in task performance. Our work demonstrates that even state-of-the-art LLMs like GPT-4 can be further improved at test-time using our simple, standalone approach.²

1 Introduction

Although large language models (LLMs) can generate coherent outputs, they often fall short in addressing intricate requirements. This mostly includes tasks with multifaceted objectives, such as dialogue response generation, or tasks with hard-to-define goals, such as enhancing program readability. In these scenarios, modern LLMs may produce an intelligible initial output, yet may benefit from further iterative refinement—i.e., iteratively mapping a candidate output to an improved one—to ensure that the desired quality is achieved. Iterative refinement typically involves training a refinement model that relies on domain-specific data (e.g., Reid and Neubig (2022); Schick et al. (2022a); Welleck et al. (2022)). Other approaches that rely on external supervision or reward models require large training sets or expensive human annotations (Madaan et al., 2021; Ouyang et al., 2022), which may not always be feasible to obtain. These limitations underscore the need for an effective refinement approach that can be applied to various tasks without requiring extensive supervision.

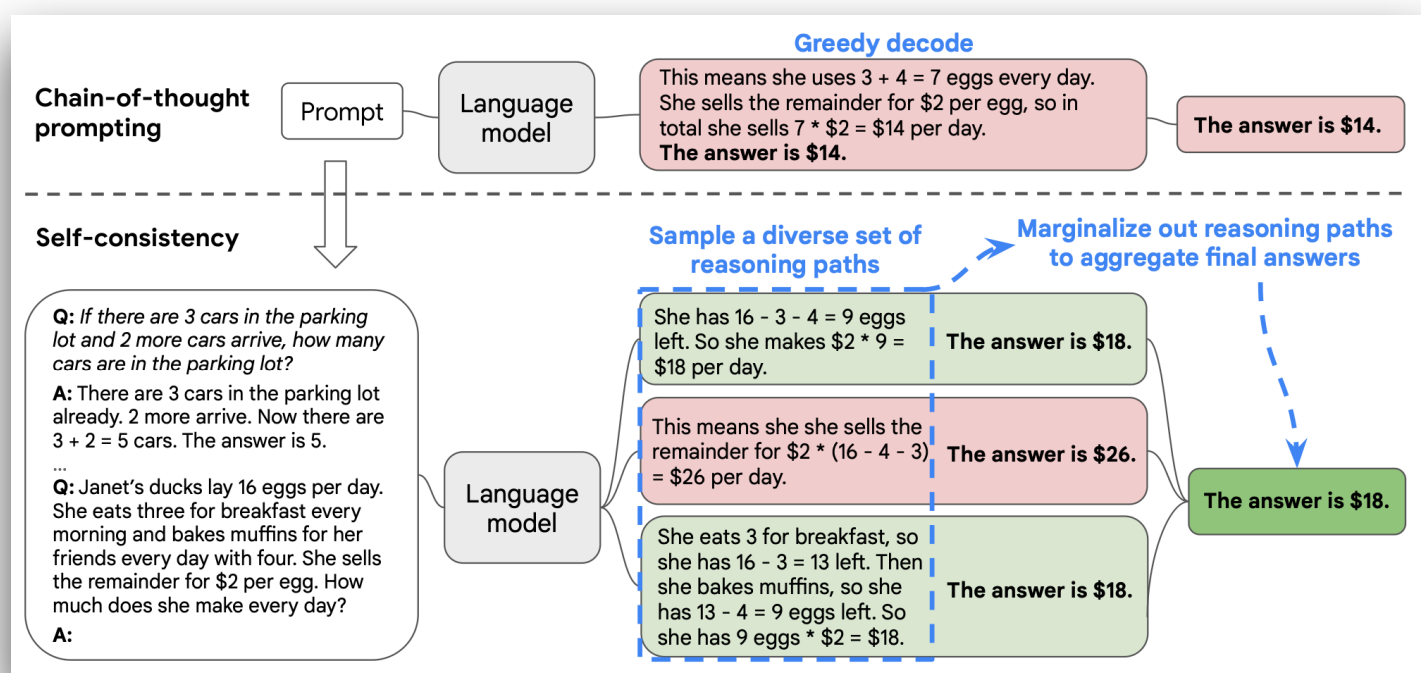
Iterative *self*-refinement is a fundamental characteristic of human problem-solving (Simon, 1962; Flugelstad et al., 2022). In this paper, we introduce SELF-REFINE, a simple, standalone approach for

Self-refinement

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

Feedback loop. A feedback loop is a mechanism in which the output of a generative model is evaluated and used to influence subsequent prompts, actions, or decisions within the system.



Instead of generating one response, the model generates **multiple reasoning paths** and picks the most consistent answer. Instead of following just one reasoning chain (which may be flawed), the model **samples multiple thought processes** and **averages them**.

SELF-CONSISTENCY IMPROVES CHAIN OF THOUGHT REASONING IN LANGUAGE MODELS

Xuezhi Wang^{†‡} Jason Wei[†] Dale Schuurmans[†] Quoc Le[†] Ed H. Chi[†]
Sharan Narang[†] Aakanksha Chowdhery[†] Denny Zhou^{†§}
[†]Google Research, Brain Team
[‡]xuezhiw@google.com, [§]dennyzhou@google.com

ABSTRACT

Chain-of-thought prompting combined with pre-trained large language models has achieved encouraging results on complex reasoning tasks. In this paper, we propose a new decoding strategy, *self-consistency*, to replace the naive greedy decoding used in chain-of-thought prompting. It first samples a diverse set of reasoning paths instead of only taking the greedy one, and then selects the most consistent answer by marginalizing out the sampled reasoning paths. Self-consistency leverages the intuition that a complex reasoning problem typically admits multiple different ways of thinking leading to its unique correct answer. Our extensive empirical evaluation shows that self-consistency boosts the performance of chain-of-thought prompting with a striking margin on a range of popular arithmetic and commonsense reasoning benchmarks, including GSM8K (+17.9%), SVAMP (+11.0%), AQuA (+12.2%), StrategyQA (+6.4%) and ARC-challenge (+3.9%).

1 INTRODUCTION

Although language models have demonstrated remarkable success across a range of NLP tasks, their ability to demonstrate reasoning is often seen as a limitation, which cannot be overcome solely by increasing model scale (Rae et al., 2021; BIG-bench collaboration, 2021, *inter alia*). In an effort to address this shortcoming, Wei et al. (2022) have proposed *chain-of-thought prompting*, where a language model is prompted to generate a series of short sentences that mimic the reasoning process a person might employ in solving a task. For example, given the question “If there are 3 cars in the parking lot and 2 more cars arrive, how many cars are in the parking lot?”, instead of directly responding with “5”, a language model would be prompted to respond with the entire chain-of-thought: “There are 3 cars in the parking lot already. 2 more arrive. Now there are $3 + 2 = 5$ cars. The answer is 5.”. It has been observed that chain-of-thought prompting significantly improves model performance across a variety of multi-step reasoning tasks (Wei et al., 2022).

In this paper, we introduce a novel decoding strategy called *self-consistency* to replace the greedy decoding strategy used in chain-of-thought prompting (Wei et al., 2022), that further improves language models’ reasoning performance by a significant margin. Self-consistency leverages the intuition that complex reasoning tasks typically admit multiple reasoning paths that reach a correct answer (Stanovich & West, 2000). The more that deliberate thinking and analysis is required for a problem (Evans, 2010), the greater the diversity of reasoning paths that can recover the answer.

Figure 1 illustrates the self-consistency method with an example. We first prompt the language model with chain-of-thought prompting, then instead of greedily decoding the optimal reasoning path, we propose a “sample-and-marginalize” decoding procedure: we first *sample* from the language model’s decoder to generate a *diverse* set of reasoning paths; each reasoning path might lead to a different final answer, so we determine the optimal answer by *marginalizing out* the sampled reasoning paths to find the most consistent answer in the final answer set. Such an approach is analogous to the human process of considering multiple hypotheses and then selecting the most likely one. After confidence is established, the model can avoid the stochasticity of the greedy decoding process.

Self-consistency

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The ‘Orchestration’ Layer

Feedback loop. A feedback loop is a mechanism in which the output of a generative model is evaluated and used to influence subsequent prompts, actions, or decisions within the system.

Meta-Prompting is a technique where the **LLM rewrites or optimizes its own prompt** to improve the quality of its responses. Instead of relying on a static prompt, the model **self-improves** its instructions before generating an answer.

MetaPrompting: Learning to Learn Better Prompts

Yutai Hou*, Hongyuan Dong*, Xinghao Wang, Bohan Li, Wanxiang Che†
Research Center for Social Computing and Information Retrieval
Harbin Institute of Technology, China
{ythou, hydong, xhwang, bhli, car}@ir.hit.edu.cn

Abstract

Prompting method is regarded as one of the crucial progress for few-shot nature language processing. Recent research on prompting moves from discrete tokens based “hard prompts” to continuous “soft prompts”, which employ learnable vectors as pseudo prompt tokens and achieve better performance. Though showing promising prospects, these soft-prompting methods are observed to rely heavily on good initialization to take effect. Unfortunately, obtaining a perfect initialization for soft prompts requires understanding of inner language models working and elaborate design, which is no easy task and has to restart from scratch for each new task. To remedy this, we propose a generalized soft prompting method called MetaPrompting, which adopts the well-recognized model-agnostic meta-learning algorithm to automatically find better prompt initialization that facilitates fast adaptation to new prompting tasks. Extensive experiments show MetaPrompting tackles soft prompt initialization problem and brings significant improvement on four different datasets (over 7 points improvement in accuracy for 1-shot setting), achieving new state-of-the-art performance.

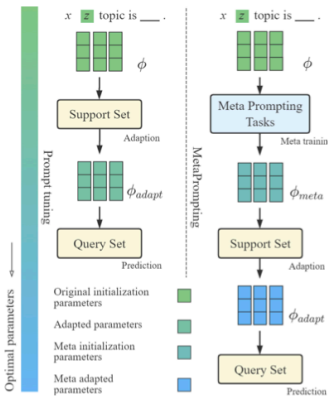


Figure 1: Comparison between conventional soft-prompting method (left) and proposed MetaPrompting (right). x denotes the query sentence, and z is learnable pseudo tokens in soft prompts. ϕ represents all trainable parameters. MetaPrompting exploits optimization-based meta-learning to find an initialization ϕ_{meta} that facilitates better and faster adaptation to new tasks.

1 Introduction

Enabling models to learn from a few labeled examples, i.e., Few-Shot Learning (FSL), is one of the key steps toward more human-like artificial intelligence. Recently, taking advantage of large-scale Pretrained Language Models (PLM) (Brown et al., 2020), prompting-based methods achieve impressive results for few-shot learning of Natural Language Processing (NLP) (Gao et al., 2021; Liu et al., 2021a; Zhao et al., 2021).

Prompting-based methods insert a piece of text, i.e. prompts, to the input examples so that the few-shot task can be solved.

modeling problem. For example, say we want to classify the sentiment of the book review “I will never read it again.”, we can append a prompt “It was” to the sentence, getting “I will never read it again. It was”. It is natural to expect a higher probability from the PLM to generate “terrible” than “great” then. Such converting bridges the gap between pre-training and target tasks. Consequently, it has better transferability and less dependence on target task data.

The performance of prompting methods is found to be greatly affected by the design of prompts (Gao et al., 2021). The design of prompts significantly affects the performance of the model.

Self-optimization

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

Feedback loop. A feedback loop is a mechanism in which the output of a generative model is evaluated and used to influence subsequent prompts, actions, or decisions within the system.

Step 1: Self-Optimizing Prompt

"Before answering, refine my question to make it more precise and useful."

◆ Model's Response (Refined Prompt):

"Summarize the most impactful advances in quantum computing from 2022–2024, focusing on breakthroughs in quantum error correction and superconducting qubits."

Step 2: Use the Improved Prompt to Generate a Better Answer

"Now, based on the refined prompt, generate the summary."

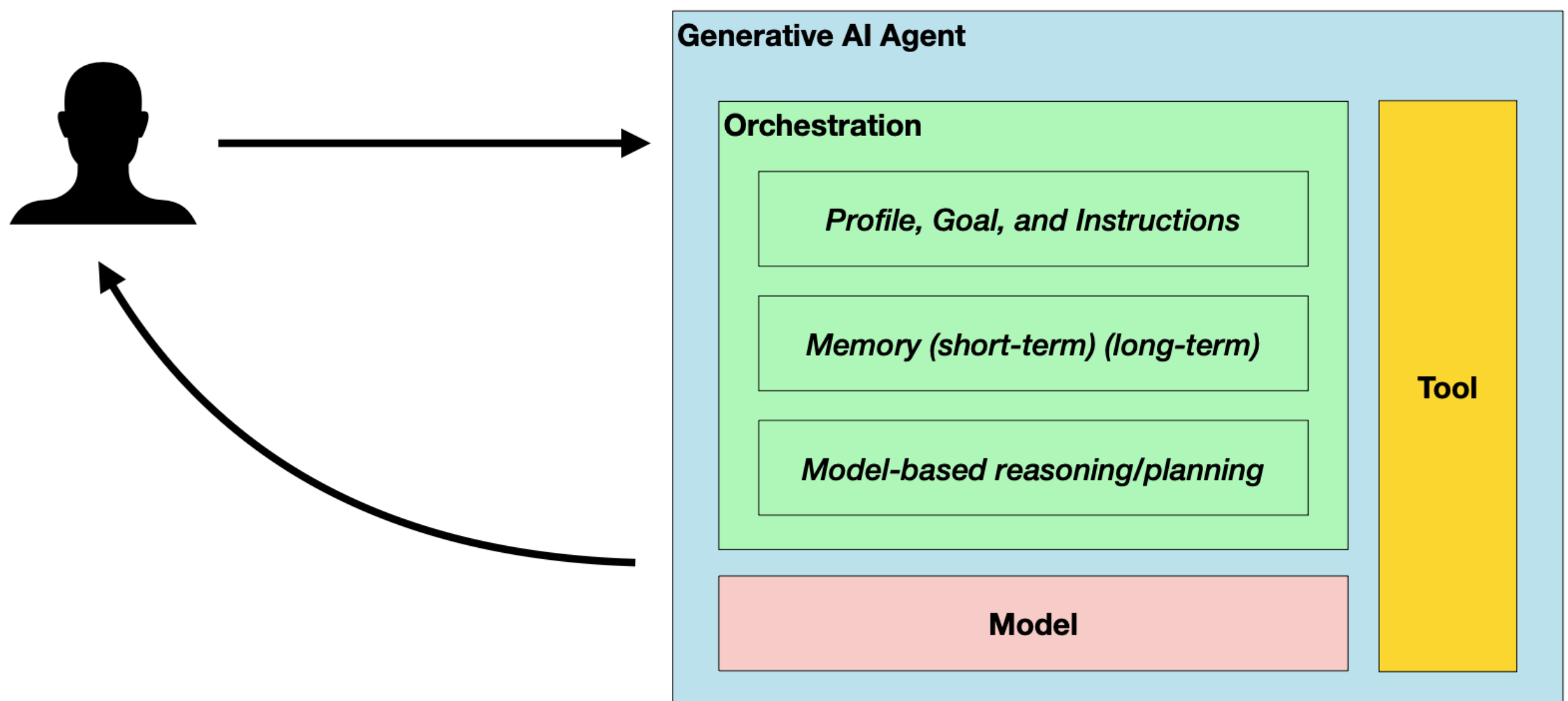
◆ Model's Final Output (Improved Answer):

"In recent years (2022–2024), quantum computing has made significant strides, particularly in the field of quantum error correction. Notably, Google's Quantum AI team introduced a scalable error correction technique that significantly reduces decoherence, while IBM unveiled its 433-qubit processor, advancing superconducting qubit stability..."

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.



Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

Orchestration. The orchestration layer describes a cyclical process that governs how the agent takes in information, performs some internal reasoning, and uses that reasoning to inform its next action or decision.

Prompt Engineering. The process of writing, refining, and optimizing inputs to encourage generative AI systems to create specific, high-quality outputs.

Memory. The architectural mechanism that enables the system to retain, retrieve, and use information across interactions to support goal-directed behavior over time.

A generative model, by itself, is **stateless**. GenAI agents are **stateful** systems.

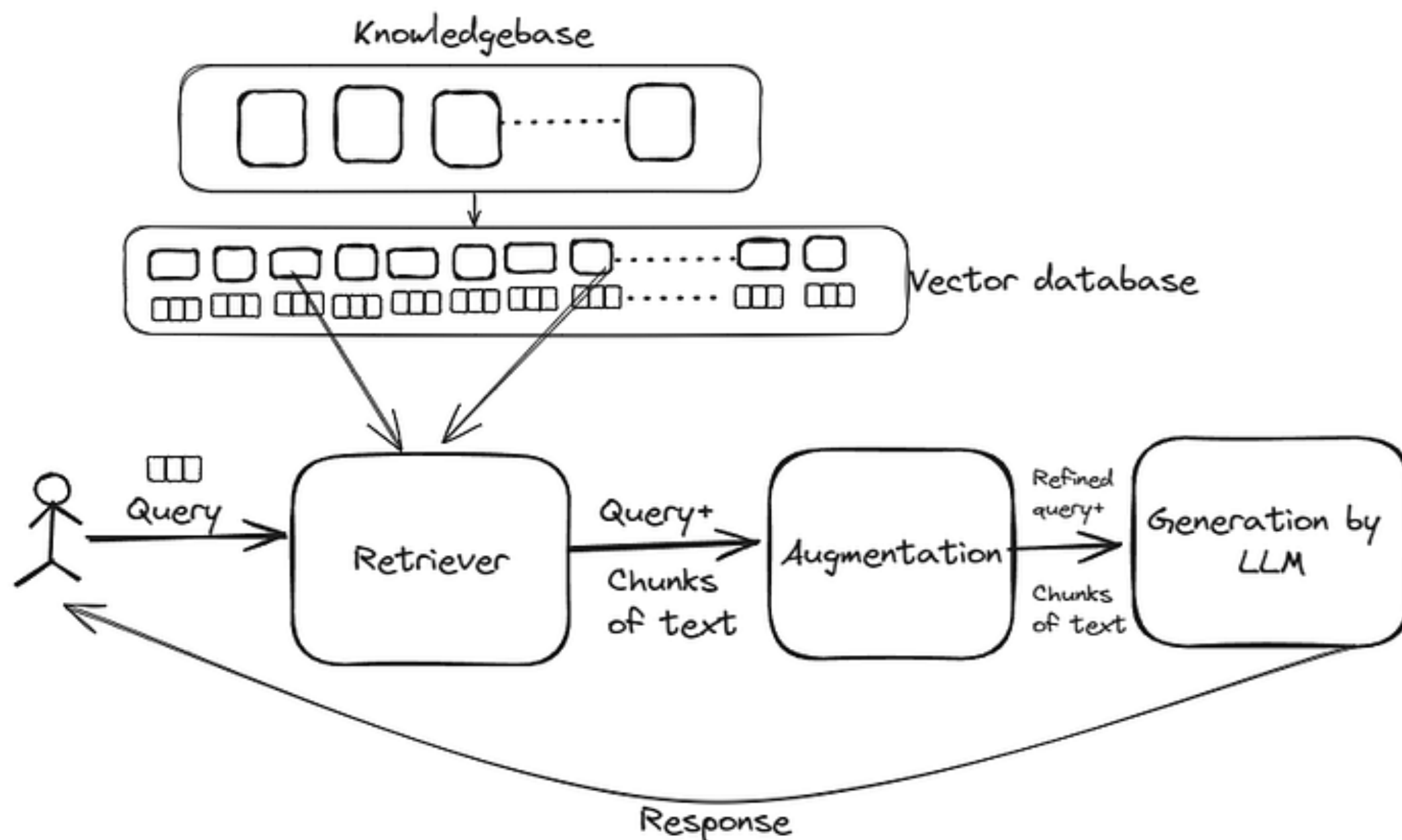
But be careful: having memory *does not mean having knowledge*. Model knowledge is fixed at training time, is incomplete, and potentially outdated. Long-term memory should not store *everything*, as raises privacy and scalability issues.

We need a way to give agents access to external knowledge without retraining models or permanently storing everything in memory.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

Retrieval Augmented Generation. This is an architectural pattern in which a generative model is augmented with an external retrieval component that supplies relevant information at inference time.



Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

Retrieval Augmented Generation. This is an architectural pattern in which a generative model is augmented with an external retrieval component that supplies relevant information at inference time.

So, RAGs should not be confused with memory - they serve a complementary task. Memory is the *system-level mechanism that preserves and reuses information about past interactions, decisions, or state* in order to influence future behavior.

Retrieval-Augmented Generation injects external knowledge into a stateless model at inference time, without modifying or retraining the model.

In other terms, RAGs are used to augment GenAI models' capabilities by exploiting external resources; memory augments GenAI models' capabilities by exploiting previous interactions and states.

By definition, memory is therefore not the entire knowledge base, raw documents, encyclopedic information, or training data.

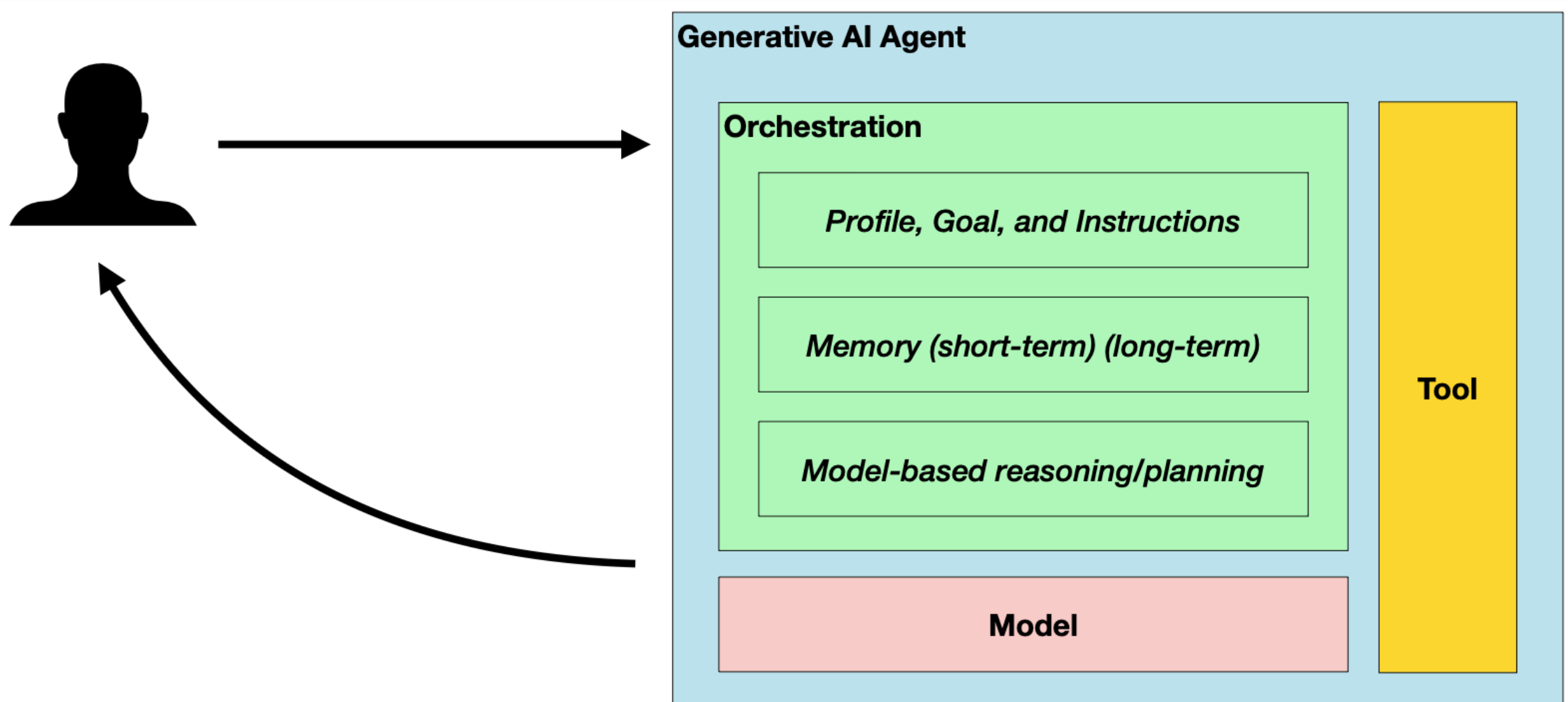
When talking about GenAI Agents, memory is a fundamental component: indeed, when an agent has memory, architectural choices determine how long information persists, how it is reused, and how it influences future behavior.

This has an effect on multiple non-functional requirements.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Tool' Layer

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.



Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Tool' Layer

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

Orchestration. The orchestration layer describes a cyclical process that governs how the agent takes in information, performs some internal reasoning, and uses that reasoning to inform its next action or decision.

Prompt Engineering. The process of writing, refining, and optimizing inputs to encourage generative AI systems to create specific, high-quality outputs.

Memory. The architectural mechanism that enables the system to retain, retrieve, and use information across interactions to support goal-directed behavior over time.

Tools. Tools empower agents to interact with external data and services while unlocking a wider range of actions beyond that of the underlying model alone. Tools can take a variety of forms and have varying depths of complexity, but typically align with common web API methods like GET, POST, PATCH, and DELETE.

Tools define what an agent *can* do, as a set of delegated interfaces. You may consider them as a kind of plug-ins that make agents more effective.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Tool' Layer

Tools define what an agent *can* do, as a set of delegated interfaces. You may consider them as a kind of plug-ins that make agents more effective.

Information & Retrieval Tools

These tools expand what the agent *can observe*. Examples can be search/browsing tools, RAG retrievers, database queries, or file readers. So, they **extend** perception and **bound** access to knowledge.

Action & Execution Tools

These tools allow the agent *to affect the world*. Examples can be code execution, file modification, API calls to external systems, or workflow triggers. So, they allow **real-world consequences**.

Control & Governance Tools

These tools bound the agent to what it *may do*. Examples can be input validation, output filtering, permission checks, human-in-the-loop approval, or policy enforcement services. So, they **constrain** and **govern** the agent.

These latter are particularly interesting, as they act *on the agent*, not *for the agent*.

Guardrails. Architectural mechanisms that constrain, monitor, and govern an agent's behavior by controlling how it observes, decides, and acts within a system.

In other terms, guardrails define the **safe operating boundaries** of an agent. Remember: if an agent has real-world consequences, then it becomes safety critical! We will come back on guardrails when discussing responsible AI engineering.

Software Engineering for Artificial Intelligence: Architecting and Operationalizing

Before leaving... What Should You Remember?

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

We All Live in a Yellow Submarine...

In the last lecture, we established a core principle of software engineering for AI-enabled systems: architecture is the binding element that guides the implementation to meet quality requirements.

Quality Requirements

Architecture / Design

Implementation

So far, we have discussed architectures for AI-enabled systems as something we design, deploy, and operate explicitly. But today, I want to argue that we are entering a phase where AI-based architectures are no longer just something we design for users. They are something we all **live inside**.

In Beatles' *Yellow Submarine*, the submarine is not the focus. It is the environment. People live in it, move inside it, and depend on it, without constantly thinking about how it works. GenAI agents are becoming exactly that kind of infrastructure.

Today, many users do not interact with a single AI model. They interact with **agent-based systems** that perceive, decide, act, and coordinate on their behalf.

Which means that GenAI agents are no longer just components in a system: they are part of the socio-technical environment.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

Introducing GenAI Agents

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

To properly understand GenAI agents, let's dissect this definition grammatically, because every word here carries architectural meaning.

"[...] using the tools"

The application uses external services, bounded capabilities, or explicitly designed interfaces. They define define **what an agent is allowed to do**.

"[...] that it has at its disposal"

Tools are provisioned, authorized, and configured. An agent does not have self-acquired capabilities and not self-expanding authority.

Agent = a composed software system that pursues explicit goals by observing a mediated environment and acting through delegated tools, without guarantees of success

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Model' Part

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

So, based on our discussion, what are the differences between a model and an agent?

Model	GenAI Agent
1. Knowledge is limited to what is available in their training data.	1. Knowledge is extended through the connection with external systems.
2. Single inference/prediction based on the user query. Unless explicitly implemented for the model, there is no management of session history or continuous context (i.e., chat history).	2. Manages session history to allow for multi-turn inference/prediction based on user queries and decisions made in the orchestration layer.
3. No native tool implementation.	3. Tools are natively implemented in agent architecture.
4. No native logic layer is implemented.	4. Native cognitive architecture that uses reasoning frameworks or other pre-built agent frameworks like LangChain.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Orchestration' Layer

GenAI Agent. In its most fundamental form, a Generative AI Agent can be defined as an application that attempts to achieve a goal by observing the world and acting upon it using the tools that it has at its disposal.

Orchestration. The orchestration layer describes a cyclical process that governs how the agent takes in information, performs some internal reasoning, and uses that reasoning to inform its next action or decision.

Prompt Engineering. The process of writing, refining, and optimizing inputs to encourage generative AI systems to create specific, high-quality outputs.

Prompt patterns. Prompt patterns are reusable, structured templates for prompts that encode proven interaction strategies between a system and a generative model.

Cognitive theories. Cognitive theories, in the context of prompt engineering, are conceptual models of human reasoning that inspire how prompts are structured to elicit desired behaviors from generative models.

Feedback loop. A feedback loop is a mechanism in which the output of a generative model is evaluated and used to influence subsequent prompts, actions, or decisions within the system.

Software Engineering for Artificial Intelligence: Architecting GenAI Agents

The Anatomy of GenAI Agents: The 'Tool' Layer

Tools define what an agent *can* do, as a set of delegated interfaces. You may consider them as a kind of plug-ins that make agents more effective.

Information & Retrieval Tools	These tools expand what the agent <i>can observe</i> . Examples can be search/browsing tools, RAG retrievers, database queries, or file readers. So, they extend perception and bound access to knowledge.
Action & Execution Tools	These tools allow the agent <i>to affect the world</i> . Examples can be code execution, file modification, API calls to external systems, or workflow triggers. So, they allow real-world consequences .
Control & Governance Tools	These tools bound the agent to what it <i>may do</i> . Examples can be input validation, output filtering, permission checks, human-in-the-loop approval, or policy enforcement services. So, they constrain and govern the agent.

These latter are particularly interesting, as they act *on the agent*, not *for the agent*.

Guardrails. Architectural mechanisms that constrain, monitor, and govern an agent's behavior by controlling how it observes, decides, and acts within a system.

In other terms, guardrails define the **safe operating boundaries** of an agent. Remember: is an agent has real-world consequences, then it becomes safety critical! We will come back on guardrails when discussing responsible AI engineering.



Software Engineering for Artificial Intelligence

Software Architectures and Deployment of GenAI-enabled Systems

